

20 RAG Techniques, Explained Simply

Retrieval-Augmented Generation = look things up first, then let the LLM answer using what it found.

STEP 1

Retrieve

Search your documents for the pieces most relevant to the question.

STEP 2

Augment

Paste those pieces into the prompt as context for the model.

STEP 3

Generate

The LLM writes an answer grounded in that context. Every variant below tweaks one of these steps.

Families: **BASICS** **STRUCTURE** **BETTER SEARCH** **SELF-CHECKING** **AGENTS & LOOPS** **RESEARCH MODELS**

Question

Retriever

Vector DB

LLM

Top-k chunks

Answer

Retrieve once, then generate

01

BASICS

Standard RAG

The classic recipe. Find the top few similar chunks in a vector database and add them to the prompt.

Query → Retrieve → Generate

Use when: simple Q&A over your docs.

Question

Keyword search (BM25)

Vector search (meaning)

Fuse & rerank

LLM

Exact words + meaning together

02

BASICS

Hybrid RAG

Mixes keyword search (exact words, like BM25) with vector search (meaning), then merges both result lists.

Keyword + Vector → Merge → Generate

Use when: names, codes or jargon must match exactly.

Full document

LLM writes context

Raw chunk

"From Acme's Q2 report, revenue section:" + original chunk

Vector index

BM25 index

Each chunk carries its own context

03

BASICS

Contextual RAG

Before indexing, an LLM adds a short note to each chunk saying where it came from and what it is about.

Chunk + Context note → Index

Use when: chunks lose meaning on their own ("it grew 3%").

Question

Vector DB

SQL database

Web search

APIs / files

Merge & rank

LLM

Search many sources, combine

04

BASICS

Multi-Source RAG

Pulls from several places at once: databases, PDFs, APIs, the web. Then combines the results.

DB + Docs + Web → Combine

Use when: knowledge is scattered across systems.

Query

Summaries

Sections

Chunks

Doc 1

\$1

Doc 2

\$2

Doc 3

\$4

Best chunks → LLM

Search broad → narrow

05

STRUCTURE

Hierarchical RAG

Indexes in layers: document summaries, then sections, then chunks. Searches from broad to specific.

Doc → Section → Chunk

Use when: you have huge, well-structured collections.

Top

Root summary

Summaries

Summary

Summary

Summary

Chunks

Cluster → summarize → repeat (a tree)

06

STRUCTURE

RAPTOR

Groups similar chunks, summarizes each group, and repeats, building a tree. Retrieval can use any level.

Cluster → Summarize → Repeat

Use when: long docs and "big picture" questions.

Question

Query v1

Query v2

Query v3

Rank Fusion (RRF)

LLM

Many phrasings → one merged ranking

07

BETTER SEARCH

Fusion RAG

Rewrites your question several ways, searches each version, then fuses the rankings (Reciprocal Rank Fusion).

1 query → N queries → Fuse

Use when: questions are vague or many-sided.

Question

Pass 1 search + draft

new query

Pass 2 search the gaps

new query

Pass 3 check & polish

Final answer

Each round learns from the last

08

BETTER SEARCH

Multi-Pass RAG

Retrieves in rounds. What it learns in one pass shapes the next search, like following clues.

Search → Learn → Search again

Use when: answers need several hops of facts.

Docs

Small LM draft 1

Small LM draft 2

Small LM draft 3

Big LLM verifies

Best answer

Draft in parallel, verify once

09

BETTER SEARCH

Speculative RAG

A small, fast model writes several draft answers from different doc subsets. A big model checks them and picks the best.

Small drafts → Big verifies

Use when: you need speed and accuracy.

Try options

Chunk size

Embeddings

Retriever

Reranker

Evaluate on test questions

Best pipeline

Auto-tunes the whole pipeline

10

BETTER SEARCH

Auto RAG

Automates the tuning: tries different chunk sizes, embeddings and rerankers, and keeps whatever scores best on tests.

Try configs → Evaluate → Pick

Use when: you don't know which setup works best.

Question

Retrieve

Evaluator grades docs

Correct refine docs

Ambiguous use both

Incorrect web search

LLM

Grade the docs → fix → generate

11

SELF-CHECKING

Corrective RAG

Grades the retrieved docs. If they are weak or wrong, it rewrites the query or falls back to web search.

Retrieve → Grade → Fix → Generate

Use when: your knowledge base is noisy or incomplete.

Question

Retrieve?

yes

Retrieve

Grade relevance

LLM

Supported? Useful?

retry if not

Answer

The model critiques its own work

12

SELF-CHECKING

Self-RAG

The model decides if it even needs to retrieve, then critiques itself: are the docs relevant, is my answer supported?

Need docs? → Answer → Self-grade

Use when: cutting hallucinations matters most.

Answer

Retriever + LLM

improves over time

User rates or corrects

Update ranking

13

SELF-CHECKING

Feedback-Based RAG

Collects user ratings and corrections, and uses them to improve ranking and answers over time.

Answer → User rates → Improve

Use when: a live product that should learn.

Clean docs

Poisoned doc

Hidden instructions

Filter & verify sources

Blocked

Safe answer

LLM

Guards against poisoned inputs

14

SELF-CHECKING

Adversarial RAG

Built to resist poisoned or misleading documents and hidden prompt injections, often tested against attacks.

Retrieve → Filter threats → Generate

Use when: sources are untrusted or security matters.

Question

Agent plans & decides

Vector DB

Web search

APIs / SQL

Good enough?

yes

Answer

no: search more

15

AGENTS & LOOPS

Agentic RAG

An AI agent plans the work: chooses which tools or sources to search, when, and loops until it is satisfied.

Plan → Use tools → Check → Loop

Use when: multi-step, open-ended tasks.

Thought

Reason + Act

Action (search / tool)

Observation (result)

Repeat until done

16

AGENTS & LOOPS

ReAct

"Reason + Act." The model alternates between thinking out loud and taking an action (like a search), reading each result.

Thought → Action → Observation

Use when: building tool-using agents.

User

clarify?

Assistant (LLM)

Knowledge

Answer

feedback

User steers the search in the loop

17

AGENTS & LOOPS

Interactive RAG

Keeps the user in the loop: asks clarifying questions and lets them steer or refine the search mid-conversation.

Ask → Clarify → Refine → Answer

Use when: research or exploratory assistants.

Input: "The [MASK] is the capital of France"

Neural retriever

Wikipedia

Knowledge-augmented encoder

Predicts "Paris"

trained together

18

RESEARCH MODELS

REALM

Google (2020). The retriever is trained together with the language model during pre-training, learning what to fetch.

Retriever + LM trained jointly

Why it matters: an early blueprint for RAG.

Question

Doc 1

Doc 2

Doc 3

LLM

LLM

LLM

Blend outputs

One LLM call per doc, then ensemble

19

RESEARCH MODELS

RePLUG

Treats the LLM as a black box. Feeds each doc separately, blends the outputs, and can tune the retriever from LLM feedback.

Doc 1..N → LLM → Ensemble

Use when: you can't modify the model (API-only).

Query

Retriever

Top docs

trained jointly

Reader (T5) Fusion-in-Decoder

Answer

Small model, strong few-shot results

20

RESEARCH MODELS

ATLAS

Meta. A retriever and a smaller language model trained together, strong on knowledge tasks with only a few examples.

Retriever ≠ Reader (joint training)

Why it matters: small model, big-model results.

Quick pick: which one do I need?

Just starting?

Standard → then Hybrid

Vague questions?

Fusion RAG

Missing exact terms?

Hybrid RAG

Bad search results?

Corrective RAG

Chunks lack context?

Contextual RAG

Too many hallucinations?

Self-RAG

Long documents?

RAPTOR or Hierarchical

Multi-step tasks?

Agentic RAG or ReAct